

ERA Tutorium 4

Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000 - 1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200 - 1

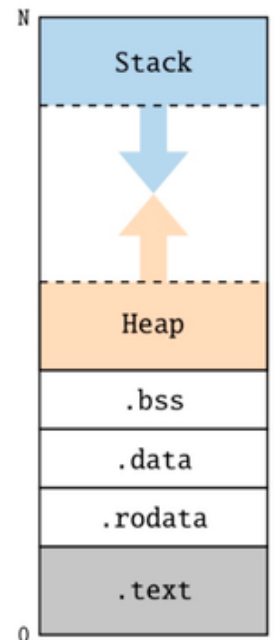
4 optional: Taschenrechner-Tester (Präsenzaufgabe 01)

Bearbeitung der Präsenzaufgabe 01 (nicht bewertet) auf <https://artemis.tum.de/courses/516>.

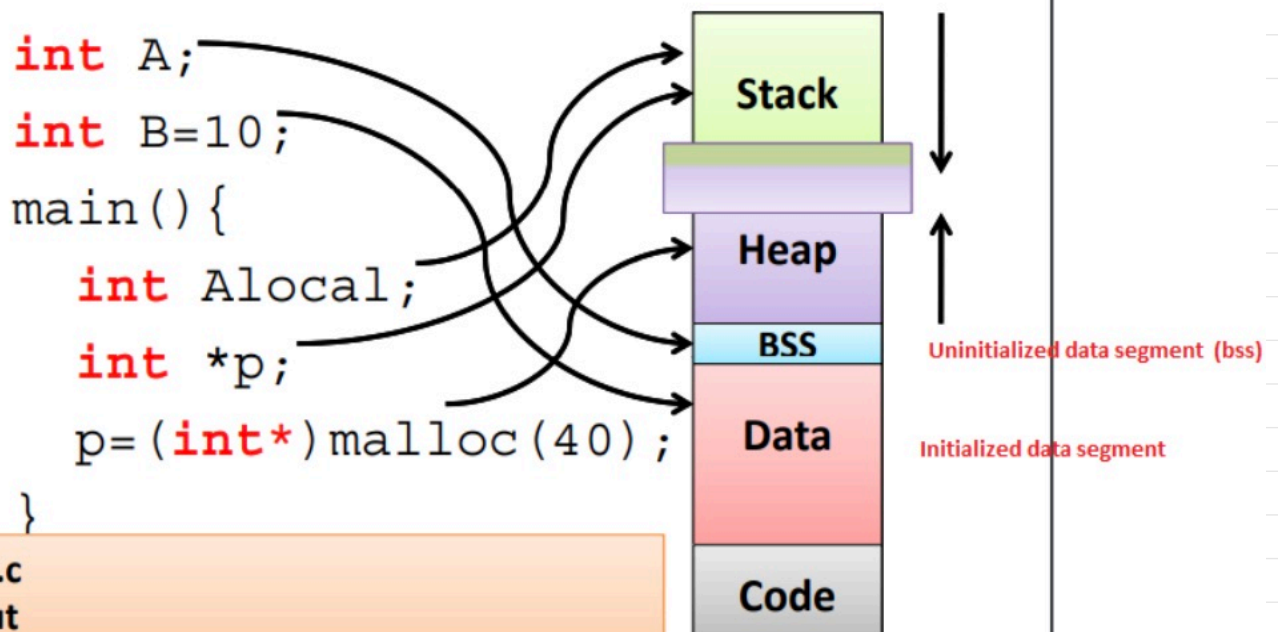
Speicherlayout

wichtige sog. Sections:

1. `.text`
beinhaltet ausführbaren Code
2. `.data`
beinhaltet globale, schreibbare, initialisierte Daten
3. `.rodata`
beinhaltet globale, nicht-schreibbare, initialisierte Daten
4. `.bss` (= block starting symbol)
beinhaltet globale, schreibbare, nicht-initialisierte Daten
effektiv nur ein Marker für „hier kommen n Bytes“



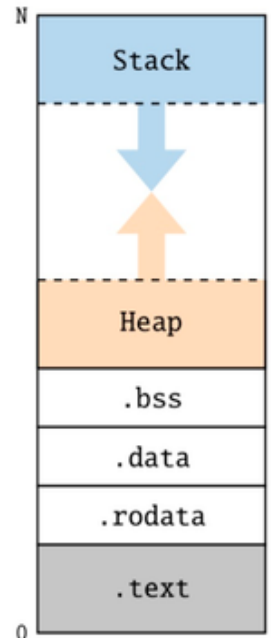
Memory layout of C program



Speicherlayout

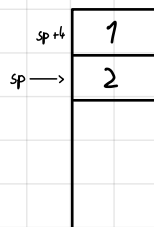
wichtige sog. Sections:

1. `.text`
beinhaltet ausführbaren Code
2. `.data`
beinhaltet globale, schreibbare, initialisierte Daten
3. `.rodata`
beinhaltet globale, nicht-schreibbare, initialisierte Daten
4. `.bss` (= block starting symbol)
beinhaltet globale, schreibbare, nicht-initialisierte Daten
effektiv nur ein Marker für „hier kommen n Bytes“



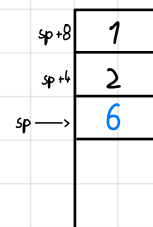
Stack:

Push $\times 5$ ($\times 5 = 6$):



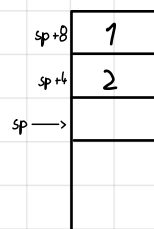
LIFO:

Pop $\times 5$:



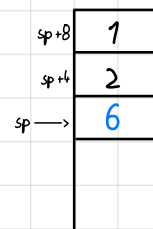
1. SP erniedrigen:

`addi sp, sp, -4`



1. Daten Laden:

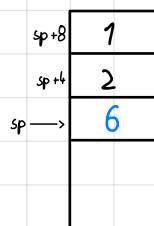
`lw  $\times 5$ , 0(sp)`



$\times 5 = 6$

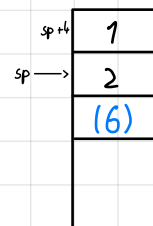
2. Daten speichern:

`sw  $\times 5$ , 0(sp)`



2. SP erhöhen:

`addi sp, sp, 4`



Calling - Convention:

1. Caller-/Callee-Saved Register

Funktion A ruft Funktion B auf.

A = caller B = callee

Callee-Saved:

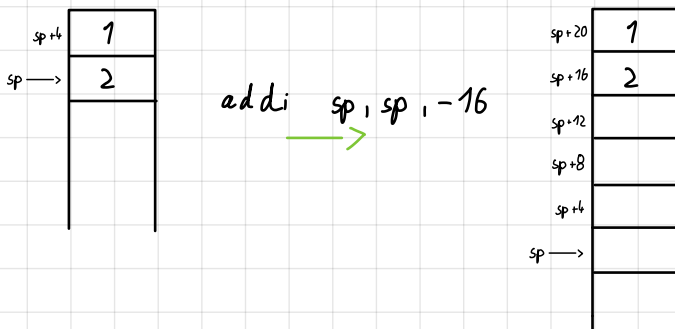
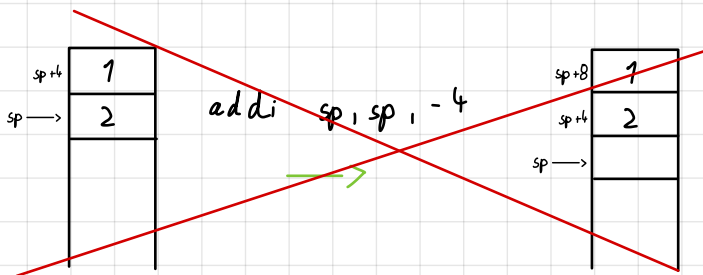
- Callee muss Werte sichern
- Bleiben für Caller unverändert

Caller-Saved:

- Callee darf Register beliebig ändern
- Caller muss Werte sichern, falls er sie später benötigt

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-7	t0-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

2. Stack-Alignment : SP Vielfaches von 16



3. Parameterübergabe

Calling Convention – Parameterübergabe

- ▶ Datentypen kleiner als 32-Bit werden auf 32-Bit extended
 - Signed Typen werden sign-extended, unsigned Typen 0-extended
- ▶ 32-Bit Werte (oder kleiner) in a-Registern, aufsteigend sortiert
- ▶ 64-Bit Werte in zwei a-Registern pro Wert, aufsteigend sortiert, untere Bit in niedrigerem Register
- ▶ Größere Datentypen (>64 Bit) werden als Pointer übergeben
- ▶ Weitere Parameter werden über den Stack übergeben
- ▶ Das erste über den Stack übergebene Argument befindet sich an Offset 0 zum Stackpointer
- ▶ Weitere Argumente werden über höhere Adressen übergeben
- ▶ Regel für Rückgabewerte: Wert wird auf gleiche Weise zurückgegeben wie er als erstes Argument übergeben worden wäre

Unterprogrammaufrufe in RISC-V

0. Register sichern, Parameter vorbereiten

- falls Daten in temporären Registern wichtig → sichern
 - z.B. in Callee-Saved Registern oder auf Stack
- gilt auch für Argumente in Registern (a0 – a7)

1. Sprung (Jump and Link)

- | | | |
|--|---|----------|
| <ul style="list-style-type: none">– jalr rd, rs, imm– jal rd, label | } | „echt“ |
| <ul style="list-style-type: none">– jar rs– jal label– j label | | |
| | } | „pseudo“ |

„and Link“ → Rücksprungadresse sichern

$PC = rs + \text{SignExt}(\text{imm}), rd = PC + 4$

$PC = PC + \text{label}, rd = PC + 4$

jalr ra, rs, 0

jal ra, label

jal zero, label

2. Operation

- sofern zutreffend: Callee-Saved Register sichern
- ???
- sofern zutreffend: Ergebnisse in a0, a1 oder Stack speichern
- sofern zutreffend: gesicherte Register wiederherstellen

3. Rücksprung (Jump (and Link) Register)

- | | | |
|--|---|----------|
| <ul style="list-style-type: none">– jalr zero, ra, 0– jr rs | } | „echt“ |
| <ul style="list-style-type: none">– ret | | |
| | } | „pseudo“ |

$PC = ra, zero = PC + 4$ (Wert verwerfen)

jalr zero, rs, 0

jalr zero, ra, 0

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schäffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

Übungsblatt 4: RISC-V Teil 3 – Rekursion und Calling Convention

10.11.2025 – 14.11.2025

Bitte beachten Sie, dass ab sofort die **Calling Convention** stets eingehalten werden muss. Details entnehmen Sie bitte der Vorlesung sowie der 4. Zentralübung.

1 Calling Convention

a) Was ist eine Calling Convention?

b) Unterteilen Sie die Register in Kategorien und klassifizieren Sie diese nach ihrem Zweck und anderen Unterschieden in der Calling Convention.

Registername(n)	Verwendung
x0/zero	Hält immer 0
ra	Hält die Rücksprungadresse
sp	Hält den Stack-Pointer
t0 – t6	Hält temporäre Werte, die nach Funktionsaufrufen nicht erhalten bleiben
s0 – s11	Hält Werte, die nach Funktionsaufrufen erhalten bleiben
a0 – a1	Hält die ersten beiden Argumente einer Funktion oder deren Rückgabewerte
a2 – a7	Hält alle verbleibenden Argumente

c) Wie können Sie zum Beispiel das Register s0 trotzdem verwenden?

d) Sie schreiben ein Programm in Assembly und rufen ein Unterprogramm auf. Welche Register sind danach garantiert unverändert? Welche müssen Sie vorher sichern um zu garantieren, dass das Programm nicht abstürzt?

e) Warum benötigt man eine Calling Convention? Was sind die Vorteile?

f) Wie werden die Parameter und Rückgabewerte in den folgenden Funktionen nach Calling Convention übergeben/zurückgegeben?

- `char add_char(char a, char b)` *a: a0 b: a1 Rückgabe: a0*
- `uint64_t add(uint64_t a, uint64_t b)` *a: a0/a1 b: a2/a3 Rückgabe: a0/a1*
- `uint64_t add(uint64_t* a, uint64_t b)` *a: a0 b: a1/a2 Rückgabe: a0/a1*
- `uint128_t copy_and_increment(uint64_t a)` *a: a1/a2 b: Rückgabe: a0*

$\hat{=}$ (uint128_t ptr, uint64_t a)*

Rekursion:

Beispiel Fakultät : $n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$ $n! = n \cdot (n-1)!$

```
// Recursive function to calculate factorial of a number
int factorial(int n) {
    // Base case: factorial of 0 is 1
    if (n == 0) {
        return 1;
    }

    // Recursive case: n * factorial of (n-1)
    return n * factorial(n - 1);
}
```

$$\begin{aligned} \text{factorial}(3) &= 3 \cdot \text{factorial}(2) \\ &= 3 \cdot 2 \cdot \text{factorial}(1) \\ &= 3 \cdot 2 \cdot 1 \cdot \text{factorial}(0) \\ &= 3 \cdot 2 \cdot 1 \cdot 1 &= 6 \end{aligned}$$

2 Rekursion in der Theorie

Gegeben sei folgendes Programm, das ab der Startadresse `0x200` im Speicher liegt. Das heißt die Bitfolge der Instruktion `addi a0, zero, 4` steht an `0x200`.

- Annotieren Sie alle Zeilen mit den entsprechenden Adressen der Instruktionen. Sie dürfen davon ausgehen, dass sie aufeinanderfolgend im Speicher liegen und keine komprimierten Instruktionen verwendet werden.
- Gehen Sie Schritt für Schritt durch die ausgeführten Befehle. Notieren Sie sich jeden Schritt, in dem sich der Stack ändert, sowie dessen Zustand zu diesem Zeitpunkt.
- Was berechnet das Unterprogramm `magic`?
- Hält das Programm die Calling Convention ein? Wenn nein, was müsste man ändern?

```

_start:
0x200    addi a0, zero, 4
0x204    jal ra, magic
0x208    ebreak

magic:
0x20C    beq a0, zero, break
0x210    addi sp, sp, -8
0x214    sw ra, 0(sp)
0x218    sw a0, 4(sp)
0x21C    addi a0, a0, -1
0x220    jal ra, magic
0x224    lw t0, 4(sp)
0x228    mul a0, a0, t0
0x22C    lw ra, 0(sp)
0x230    addi sp, sp, 8
0x234    jalr zero, 0(ra)

break:
0x238    addi a0, zero, 1
0x23C    jalr zero, 0(ra)

```

Handwritten notes:

$a0 = 24$

$t0 = 4$

$ra = 0x208$

0x0124	⋮	← sp
0x0120	0x4	
0x011C	0x208	
0x0118	0x3	
0x0114	0x224	
0x0110	0x2	
0x010C	0x224	
0x0108	0x1	
0x0104	0x224	
0x0100	⋮	

3 Rekursive Folge

Schreiben Sie ein Unterprogramm, welches diese Folge rekursiv berechnet:

$$a_n = 2 \cdot a_{n-1} + n, \quad a_0 = 10$$

$$a_1 = 2 \cdot a_0 + 1 = 2 \cdot 10 + 1 = 21$$

Verwenden Sie nur Befehle aus Abbildung 1.

Hinweis: Sie können das Ergebnis mit dieser Formel überprüfen: $a_n = -n + 3 \cdot 2^{n+2} - 2$.

4 optional: Größter gemeinsamer Teiler

Erstellen Sie ein Unterprogramm `ggT`, welches den größten gemeinsamen Teiler von zwei in `a0` und `a1` übergebenen positiven Zahlen rekursiv berechnet. Das Ergebnis soll in `a0` zurückgegeben werden. Als Hilfestellung ist unten C-Code zur rekursiven Berechnung des `ggT` gegeben. Achten Sie auch darauf die Calling Convention einzuhalten.

Hinweis: **unsigned** steht für **unsigned int**.

```
unsigned ggT(unsigned a, unsigned b) {  
    if (a==b)  
        return a;  
    else if (a < b)  
        return ggT(a, b-a);  
    else  
        return ggT(a-b, b);  
}
```

Schreiben Sie ein Unterprogramm, welches diese Folge rekursiv berechnet:

$$a_n = 2 \cdot a_{n-1} + n, a_0 = 10$$

$$a_1 = 2 \cdot a_0 + 1 = 2 \cdot 10 + 1 = 21$$

folge(n) {

if (n == 0) return 10;

return folge(n-1) * 2 + n;
}

folge:

beq zero, a0, break # Base case

addi sp, sp, -16
sw ra, 0(sp)
sw a0, 4(sp) } Register sichern

addi a0, a0, -1 # Parameter vorbereiten
jal ra, folge # Unterprogramm aufruf

lw ra, 0(sp)
lw t0, 4(sp)
addi sp, sp, 16 } Register wiederherstellen

slli a0, a0, 1
add a0, a0, t0 } $2 \cdot a_{n-1} + n$

jalr zero, 0(ra) # return

break:

addi a0, zero, 10 # $a_0 = 10$ (Basecase)
jalr zero, 0(ra) # return